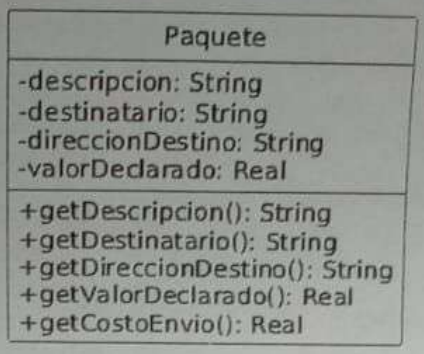
Patrones

OO2 - Parcial 7/6/2025

Ejercicio 1- Envío de paquetes

Se quiere mejorar un sistema de envíos de paquetes que permite registrar los paquetes enviados y calcular sus costos.

De los paquetes se conoce: el valor declarado, la dirección de destino, el destinatario y una descripción, como puede verse en el diagrama UML. El costo de envío es el 5% del valor declarado.



Se plantea la incorporación de servicios adicionales para personalizar la entrega, mejorando la seguridad, velocidad en el envío o manejo especial del paquete, sin modificar la clase Paquete mencionada anteriormente, pero permitiendo que la clase pueda implementar una interfaz en caso de ser necesario.

Los servicios adicionales que se quieren implementar son:

- ***Con seguro***: Incrementa el costo de envío en un 20% del valor declarado y agrega la nota *"con seguro"* a la descripción.
- ***Con seguimiento en tiempo real***: permite que el cliente siga el trayecto en tiempo real. Esto incrementa el costo de envío en 2000 pesos, pero no modifica la descripción.
- **Entrega exprés**: Incrementa el costo de envío en un 50% del valor declarado y agrega la nota "entrega express" a la descripción.
- **Manipulación frágil**: No incrementa el costo, pero agrega la nota "frágil" a la descripción.

Se pide que los servicios puedan combinarse libremente.

Por ejemplo, un paquete descrito como una "Caja de libros" con un valor declarado de $20.000 que incluye los servicios adicionales de seguro ($4.000) y entrega express ($10.000), resulta en

- Costo de envío: 5%: $ 1.000
- Seguro 20% de 20.000: $ 4.000
- Entrega express 50% de 20.000: $ 10.000
- Costo final del envío: $ 15.000

y la descripción de "Caja de libros con seguro entrega express".

Tareas (debe realizar los cuatro ítems para aprobar el tema):

1. Realice un diagrama UML de clases para su solución al problema planteado.
2. Indique claramente en el diagrama UML el o los patrones de diseño que utiliza en el modelo y el rol que cada clase cumple en cada uno.
3. Implemente su solución en Java
4. Escriba un test que muestre cómo instanciar y utilizar su solución para el caso de la "Caja de libros" con los servicios detallados anteriormente.

Refactoring

OO2 - Parcial 7/6/2025

Dado el siguiente código que implementa la generación de reportes

```
public class ReportGenerator {
  private String type;
  public ReportGenerator(String type) {  this.type = type; }

  public void generateReport(Document document) {
    if ("PDF".equals(type)) {
      // crear documento y configurar metadatos
      DocumentFile docFile = new DocumentFile();
      docFile.setTitle(document.getTitle());
      String authors = "";
      for (String author : document.getAuthors()) {
        authors += ", " + author;
      }
      docFile.setAuthor(authors);
      docFile.contentType("application/pdf");
      docFile.setPageSize("A4");

      // crear exportador y setear el contenido
      PDFExporter exporter = new PDFExporter();
      byte[] content = exporter.generatePDFFile(document);
      docFile.setContent(content);

      // guardar el documento
      this.saveExportedFile(docFile);
    } else if ("XLS".equals(type)) {
      // crear documento y configurar metadatos
      DocumentFile docFile = new DocumentFile();
      docFile.setTitle(document.getTitle());
      String authors = "";
      for (String author : document.getAuthors()) {
        authors += ", " + author;
      }
      docFile.setAuthor(authors);
      docFile.contentType("application/vnd.ms-excel");
      docFile.setSheetName(document.getSubtitle());

      // crear exportador y setear el contenido
      ExcelWriter writer = new ExcelWriter();
      byte[] content = writer.generateExcelFile(document);
      docFile.setContent(content);

      // guardar el documento
      this.saveExportedFile(docFile);
    }
```

```
class ReportGeneratorTest {
  ReportGenerator generatorPDF;
  ReportGenerator generatorXLS;
  Document document;

  @BeforeEach
  void setUp() {
    document = new Document("Informe");
    document.addAuthor("Carlos");
    document.addAuthor("Ana");

    generatorPDF = new ReportGenerator("PDF");
    generatorXLS = new ReportGenerator("XLS");
  }

  @Test
  void testPDF() {
    generatorPDF.generateReport(document);

    // aca se detallan los asserts
  }

  @Test
  void testXLS() {
    generatorXLS.generateReport(document);

    // aca se detallan los asserts
  }
```

Tareas: (debe realizar los tres ítems para aprobar el tema):

1. Enumere los code smell que encuentra en el código indicando las líneas afectadas.
2. Indique que refactorings utilizará para solucionarlos considerando que se desea incluir nuevos formatos para exportar (XLSX, CSV entre otros). Explique los pasos necesarios para realizar los refactorings elegidos, haciendo referencia al código cuando corresponda. Muestre el código final resultado luego de aplicar esos refactorings en la clase ReportGenerator y si hace falta, en la clase ReportGeneratorTest.
3. Realice el diagrama de clases del código refactorizado. Si utilizó un patrón de diseño, indíquelo en el diagrama mostrando los roles del patrón.